
ALL YOU NEED IS JSON++: DECLARATIVE INHERITANCE AND EVALUATION FOR JSON

Hiroshi Ukai
LegalOn Technologies
Shibuya, Tokyo
dakusui@gmail.com

September 11, 2025

ABSTRACT

JSON and its derivatives (YAML, TOML, HCL) are widely used for configuration and data exchange, but they lack mechanisms for structural reuse and computed values. This leads to redundancy, maintenance overhead, and ecosystem-specific workarounds such as templating or ad hoc extensions.

We introduce **JSON++**, a lightweight extension of JSON that adds two declarative constructs: **object inheritance** and **expression evaluation**. JSON++ preserves full JSON validity and follows the UNIX philosophy by focusing narrowly on structural reuse while delegating human-readability concerns to higher-level notations.

As a reference implementation, we present **jq-front**, a thin pre-processor atop jq. jq-front demonstrates how JSON++ enables expressive and maintainable pipelines without new syntax or toolchain fragmentation. Because JSON++ applies uniformly to JSON-compatible formats, it also yields YAML++, TOML++, and HCL++ “for free,” offering a unifying abstraction across ecosystems.

1 Introduction

1.1 Motivation

The proliferation of structured data formats such as JSON[1, 2], YAML[3], TOML[4], and HCL[5] has enabled software systems to describe their configuration, infrastructure, and workflows in a machine-readable and human-friendly manner. Despite their widespread adoption, these formats often lack abstraction mechanisms. Developers and operators are therefore forced to duplicate information across multiple files and projects, which hinders maintainability and reduces productivity.

This problem is widely acknowledged in practice. For instance, long-time Terraform[6] users describe facing “HCL hell”—large infrastructure repositories with repetitive configuration blocks that are difficult to refactor or abstract [7]. Similar concerns arise in the Ansible[8] ecosystem, where practitioners often struggle between duplicating tasks across playbooks and roles or introducing premature abstractions using includes, roles, or Jinja2[9] templating [10]. These trade-offs highlight a common pattern across configuration languages: either accept repetitiveness for clarity or add ad hoc syntax sugars at the cost of readability and debuggability.

Several approaches have been attempted to mitigate these challenges.

Some applications attempt to mitigate this redundancy by introducing ad hoc extension features, where users are forced to learn a similar but slightly different syntax every time they try a new application. Another family of solutions involves embedding a template engine (e.g., Handlebars, Jinja2), which allows developers to generate configuration files programmatically. However, although the template syntax resembles the host configuration language, it is essentially a distinct one, which limits the availability of mature tooling and programmatic reuse.

Yet another approach is to extend the configuration language itself, as seen in HOCON[11].

What is still missing is a lightweight and uniform mechanism to extend and transform structured data across heterogeneous environments.

1.2 Contribution

This work introduces *JSON++*, a lightweight but powerful extension of JSON. JSON++ preserves full JSON validity while adding two declarative mechanisms: inheritance across objects and evaluable string expressions for both keys and values. By focusing on structural reuse and leaving human-oriented readability to higher-level notations such as YAML or TOML, JSON++ follows the UNIX philosophy[12] of doing one thing well in a minimal and composable way.

Because many popular formats are either JSON-compatible or easily converted to JSON, providing JSON++ inherently enables YAML++, TOML++, and HCL++ as well. This illustrates the broader impact of JSON++: rather than introducing yet another bespoke syntax, it supplies a minimal core that can enhance existing notations uniformly.

jq-front is presented as the current reference implementation of JSON++. Implemented as a thin preprocessor atop *jq*, it demonstrates how these abstractions can be applied in practice, showing that existing JSON ecosystems can gain expressive power without adopting a new syntax or breaking compatibility. Because many popular formats are compatible with JSON—either natively or through straightforward conversion—*jq-front* enables a uniform pipeline where diverse notations can benefit from JSON++ enhancements and still produce plain JSON as output.

Beyond enabling YAML++, TOML++, and HCL++ through a uniform extension of JSON, JSON++ addresses a broader challenge in practice: the proliferation of *ad hoc* enhancements within individual ecosystems (see Section 1.1). Tools such as Terraform[6], Ansible[8], or Helm[13] each introduce their own extension mechanisms—interpolation, templating, includes—that are similar in spirit but mutually incompatible. This fragments developer knowledge and complicates reuse across projects. JSON++ remedies this situation by providing a lightweight, application-agnostic mechanism for structural reuse and evaluation. Because its output remains plain JSON, the same abstractions apply uniformly across the JSON family without the need for tool-specific syntactic sugars.

In the remainder of this paper, we present the design of *jq-front*, illustrate its use cases, and discuss its implications for software engineering practice.

2 Features

The design of *jq-front* and JSON++ is guided by two core ideas:

- JSON is the lingua franca of structured data.
- Developers need a way to define JSON more expressively, without duplicating large structures or hard-coding values.
- Tools should follow the UNIX philosophy: small, composable, and interoperable with other command-line utilities. This allows users to define their own DSL based on well known notations such as YAML by building a processing pipeline with its pre-defined files.

Building on these ideas, *jq-front* provides the following features:

2.1 JSON Inheritance Mechanism

jq-front provides a JSON inheritance mechanism that allows new objects to be defined in terms of existing ones. Unlike inheritance in object-oriented programming, which establishes type hierarchies and polymorphic behavior, *jq-front*’s inheritance is declarative and merge-based. The result of inheritance is always plain JSON, suitable for immediate use in existing pipelines.

jq-front supports **two inheritance modes**:

- `$extends` (*parent-first*): The base object(s) are expanded first, and the local definition selectively overrides conflicting fields. This mode is particularly useful when a pipeline provider distributes a set of predefined baseline files (e.g., default configurations), and local files need to extend them by overriding only what differs.
- `$includes` (*child-first*): The local definition is applied first, and parent objects fill in missing fields or override them in reverse order. This mode is useful when the local file provides only a fragment of the final output and external structure is provided by the pipeline.

This dual mechanism gives users explicit control over conflict resolution strategies, allowing them to choose between preserving parent defaults or privileging local values.

Inheritance operates with the following properties:

- **Selective overriding:** Only fields that differ need to be specified, reducing redundancy.
- **Deep merging:** Nested structures are merged recursively, enabling extension of complex configurations without rewriting them.
- **Multiple parents:** Objects may inherit from more than one base, with deterministic rules governing merge order.

Together, these features make it possible to maintain large collections of JSON files (e.g., configuration sets, test cases, deployment descriptors) with minimal duplication, while still retaining control over how definitions are combined.

2.2 Value Evaluation from Expressions

Instead of static values, jq-front lets nodes be defined by evaluating a string expression associated with the path.

- **Expression language:** jq-front leverages the power of Bash and jq to compute values dynamically.
- **Context-aware computation:** Expressions can reference other fields, external and built-in symbols (i.e., variables or functions).
- **Deterministic output:** Each evaluated node is replaced with its resolved value, producing standard JSON as output.

This makes it possible to encode rules, parameterized values, or environment-dependent variations directly in JSON.

In addition to evaluating values, jq-front also supports **key evaluation from expressions**. This feature allows JSON field names themselves to be computed dynamically, enabling structures where both keys and values can adapt to context. Key evaluation is particularly useful when schema fragments must be generated programmatically, such as creating environment-specific field names.

3 Design

The design of jq-front builds on two principles: (1) extensibility of JSON objects through inheritance-like mechanisms, and (2) dynamic value evaluation via embedded expressions. These principles are realized by combining JSON as the base representation with jq as the evaluation engine. This section outlines the architectural components, internal workflow, and design choices.

3.1 Architectural Overview

jq-front is a command-line wrapper around jq that introduces a lightweight pre-processing stage. The tool accepts JSON documents augmented with special annotations, expands these into standard JSON, and then invokes jq for transformation or querying. The architecture consists of two parts, each of which has internal steps (Figure 1):

- Inheritance mechanism
- Node processing mechanism

One thing to be noted is that those two mechanisms are completely separated. That is, at first, all the related JSON files are overlain in a specified order. After this step, all the keys and values that need to be evaluated become resolved. Then, each key in the file will be processed one by one from top to bottom in dictionary order of JSON paths. The same will be repeated on the value side.

3.2 Inheritance Mechanism

The inheritance mechanism is structured around three elements: file-level inheritance, local-node materialization, and node-level inheritance. Inheritance mechanism is designed so that each file's inheritance is resolved in bottom-up manner. This ensures inherited files are in a state where their inheritances are always resolved in every file's inheritance resolution.

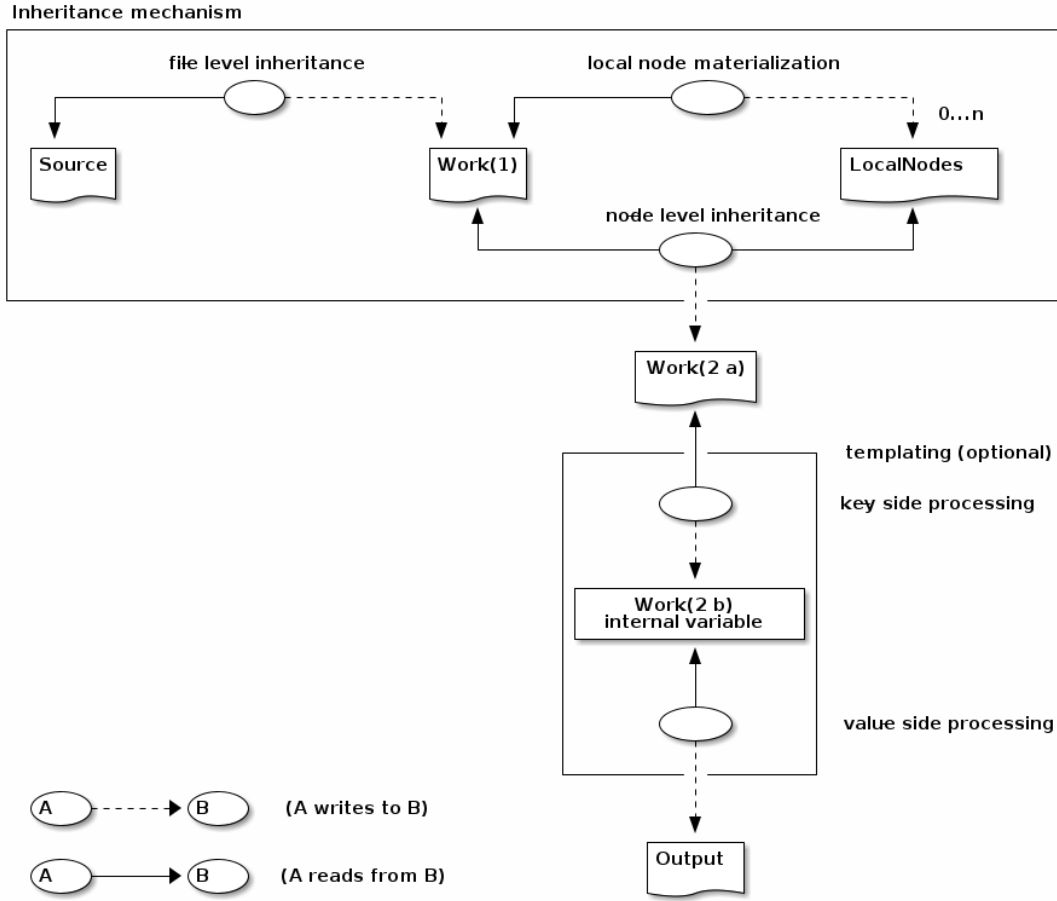


Figure 1: Overview of the processing pipeline

3.2.1 File-Level Inheritance

File-level inheritance allows a JSON file to declare other files as its parents. The pre-processor expands the parent files first and then merges them into the local file according to a specified strategy. This is the most common inheritance mode, suitable for composing large configurations where baseline definitions are extended or overridden locally. File-level inheritance is declared by referencing file names directly under `$extends` or `$includes`.

3.2.2 Local Node Materialization

For nodes to be reused within a file, creating external files is unnecessary and JSON++ allows to define such nodes under `$local` element at the root level. Those nodes are placed as files in a temporary directory, from which node-level inheritance searches nodes to be inherited. The temporary directory is created for each file which declares `$local` element. Thus, it ensures a proper local node is inherited even if multiple files involved in the inheritance mechanism are defining nodes with the same name locally.

3.2.3 Node-Level Inheritance

In addition to entire files, individual nodes inside a JSON document can also inherit from other nodes. This mechanism supports intra-document reuse and avoids repetition of deeply nested structures. Node-level inheritance is resolved recursively, with deterministic rules for conflict resolution. This feature makes it possible to structure large configuration files with shared fragments without breaking them into many files.

3.3 Node Processing Mechanism

Node-Level Processing Mechanism consists of two stages. One is key side and the other is value-side. Value-side processing is the typical use-case, which is executed after key-side's.

3.3.1 Value-Side Processing

Value-side evaluation allows a node's value to be defined by an expression rather than a literal. Expressions are written as strings prefixed with a marker (e.g., `eval:`) and are evaluated against the current JSON context. This makes it possible to compute derived values, parameterize configurations, or encode environment-dependent logic directly in JSON. For example, a field can be defined as `"double": "eval:number:$(($(ref .base) * 2))"`, which resolves deterministically into a standard JSON value.

3.3.2 Key-Side Processing

In addition to values, JSON++ also allows keys to be computed dynamically. This enables environment-specific prefixes is created on the fly. Key-side evaluation broadens the applicability of JSON++ to scenarios where the structure itself must be parameterized.

3.4 Workflow Integration

Together, these mechanisms form a coherent workflow. A typical pipeline consists of writing a JSON++ document with inheritance and evaluation directives, preprocessing it with `jq-front`, and passing the resulting plain JSON to downstream tools. By separating concerns across file-level and node-level inheritance, shell-output inheritance, and value- and key-side evaluation, `jq-front` demonstrates that JSON++ provides a minimal but complete set of abstractions for structured reuse.

4 Examples

This section illustrates how `jq-front` can be used to construct and transform JSON objects through inheritance and evaluation. We follow the examples provided in the documentation.

In this section, we present compact and clean examples to clarify the mechanics of JSON++. In practice, however, configurations are far more verbose and repetitive. A more realistic, large-scale example is provided in Appendix A.

4.1 File-Level Inheritance

Consider the following base object `A.json`:

```

1 {
2   "name": "A",
3   "value": 1
4 }
```

We can extend this object by defining `B.json`:

```

1 {
2   "$extends": ["A.json"],
3   "name": "B"
4 }
```

Here, `B.json` inherits all fields from `A.json` while overriding the `name` attribute. Evaluating this file with `jq-front` produces:

```

1 {
2   "name": "B",
3   "value": 1
4 }
```

This demonstrates how `jq-front` eliminates duplication by allowing structured reuse of existing definitions.

4.2 Multiple Inheritance

jq-front also supports multiple inheritance. For instance, `AA.json` extends both `A.json` and `B.json`:

```
1 {
2   "$extends": ["A.json", "B.json"],
3   "name": "AA"
4 }
```

The result merges fields from both parents, applying the standard resolution rules:

```
1 {
2   "name": "AA",
3   "value": 1
4 }
```

This ability to compose objects declaratively is particularly powerful when working with large configuration files.

4.3 Local Nodes and Node-Level Inheritance

It is frequent to reuse the same object or quite similar node multiple times within single JSON file in practice. In such a case you can define "local nodes" under a key `$local`. Following is an example.

```
1 {
2   "$local": {
3     "A": {
4       "name": "A",
5       "value": 1
6     },
7     "B": {
8       "name": "B"
9     }
10  }
11  "internalNode": {
12    "$extends": ["A", "B"],
13    "name": "AA"
14  }
15 }
```

You can reuse the defined local nodes at any path but the root level in the same object. This is an intended design to avoid circular dependencies.

The file above will result in the output shown below.

```
1 {
2   "internalNode": {
3     "name": "AA",
4     "value": 1
5   }
6 }
```

4.4 Evaluated Values

Beyond inheritance, jq-front supports evaluation of string expressions. In `T.json`, values are defined dynamically:

Listing 1: Node value evaluation: Input

```
1 {
2   "base": 10,
3   "double": "eval:${(($ref .base) * 2)}",
```

```

4   "nested": {
5     "triple": "eval:${(($ref .base) * 3)}"
6   }
7 }

```

When processed with jq-front, the string values prefixed by `eval:` are evaluated:

Listing 2: Node value evaluation: Output

```

1 {
2   "base": 10,
3   "double": 20,
4   "nested": {
5     "triple": 30
6   }
7 }

```

This mechanism enables compact expression of derived values without resorting to external templating tools.

5 Artifact Availability

For reproducibility and demonstration purposes, jq-front is distributed as a Docker image. Running jq-front inside a container ensures all dependencies are managed automatically and avoids platform-specific issues. The Dockerfile is available on GitHub together with the source code and examples. This allows readers to reproduce the examples in this paper with a single command, without manual setup.

```

1 function jq-front() {
2   docker run --rm -i \
3     -v "${HOME}:/var/lib/jf/${HOME}" \
4     -v "${HOME}/.jq-front.rc:/root/.jq-front.rc" \
5     -e JF_PATH_BASE="/var/lib/jf" \
6     -e JF_PATH="${JF_PATH}" \
7     -e JF_DEBUG="${JF_DEBUG:-disabled}" \
8     -e JF_CWD="$(pwd)" \
9     dakusui/jq-front:"${JF_DOCKER_TAG:-v0.57}" "${@}"
10 }

```

The repository[14] also provides documentation of environment variables (e.g., `JF_PATH`) and command-line options for advanced usage.

6 Discussion

Configuration languages have long faced a central tension between human-readability and structural expressiveness. Formats such as HOCON (Human-Optimized Config Object Notation) attempt to solve this by enriching JSON-like syntax with features such as comments, concatenation, substitutions, and type flexibility. These additions are designed to improve readability and reduce verbosity, but they couple human ergonomics with machine semantics into a single notation.

jq-front adopts a different philosophy. It treats human-readability and structural expressiveness as orthogonal concerns, consistent with the UNIX philosophy of “do one thing well.” The two minimal extensions introduced by JSON++ — inheritance (`$extends`, `$includes`) and expression evaluation (for both values and keys) — provide structural power while preserving full JSON validity. Readability, when needed, can be layered on later through format conversion (e.g., YAML or TOML), while jq-front itself remains focused on machine-processable expressiveness.

An important consequence is that by defining JSON++, we automatically obtain YAML++, TOML++, and HCL++ as well. Because these formats are either natively compatible with JSON or easily converted to it, they can all benefit from the same abstraction mechanisms without new parsers or syntactic extensions. This contrasts with HOCON, which introduces its own ecosystem-specific notation and parsing model. JSON++ therefore acts as a unifying abstraction layer for the entire JSON family, delivering expressive power without fragmenting the tool landscape.

Aspect	jq-front (JSON++)	HOCON	Jsonnet	CUE
Core abstraction	Inheritance + evaluation (valid JSON)	Includes, substitutions, concat	Data templating language	Constraint language + validation
Primary focus	Structural expressiveness	Human readability	Expressive generation	Validation + composition
Integration model	UNIX pipelines; format-agnostic	JVM config notation	Standalone interpreter	Standalone toolchain

Table 1: Comparison of jq-front/JSON++ with HOCON, Jsonnet, and CUE.

Beyond HOCON, two other systems are worth comparison: Jsonnet and CUE. Jsonnet [15] introduces a small programming language designed for data generation, including functions, comprehensions, and mixins. This makes it highly expressive, but Jsonnet files are not valid JSON and require a separate toolchain and language surface to learn. jq-front, by contrast, deliberately avoids introducing a new language layer: it keeps JSON as the authoring format and delegates computation to jq, thereby preserving interoperability with existing JSON tooling.

CUE [16] emphasizes constraints and validation, combining data description with schema checking in a single declarative language. This approach yields strong guarantees and reduces redundancy through unification, but again requires adopting a dedicated syntax and mental model. jq-front diverges here by focusing narrowly on reuse and evaluation, leaving validation concerns to established mechanisms such as JSON Schema. In this way, jq-front complements rather than replaces schema-based systems.

Together, these comparisons position JSON++/jq-front at a minimalist point in the design space: it retains JSON validity at rest, enables declarative inheritance and evaluation, and stays composable within UNIX pipelines. Jsonnet offers maximal computation, CUE emphasizes constraints and validation, and HOCON balances human readability with expressiveness. jq-front’s distinctive contribution is to supply structural abstraction without leaving the JSON ecosystem or introducing a new configuration language.

6.1 Illustrative Example

Consider the following example written in jq-front:

Listing 3: jq-front example with inheritance and evaluation

```

1 {
2   "$extends": ["base.json"],
3   "service": {
4     "port": "eval:number:$(ref .base.port)"
5   }
6 }
```

Because both the input and output of jq-front are valid JSON, any notation that can be converted to JSON can benefit from jq-front’s enhancements without requiring changes to downstream processors. The equivalent in HOCON looks as follows:

Listing 4: Equivalent HOCON example

```

1 include "base.conf"
2
3 service {
4   port = ${base.port}
5 }
```

This highlights a key distinction. HOCON attempts to address both human-readability and structural expressiveness within a single monolithic notation, whereas jq-front focuses solely on expressiveness. Readability is left to higher-level notations, allowing jq-front to remain minimal and composable within JSON pipelines.

6.2 Performance and Limitations

From a performance perspective, jq-front’s current implementation leverages Bash and jq. This layered approach is deliberately simple and sufficient for moderate-scale use, but it may impose overhead in deeply nested or large

configurations. Unlike HOCON’s single-language parser, jq-front orchestrates multiple layers of processing. This makes it slower in raw execution speed, but more flexible and easier to integrate into diverse pipelines.

Although jq-front is typically run server-side in controlled pipelines, the use of shell-based `eval` means that incorporating untrusted expressions or includes still poses an injection risk, which should be mitigated with trusted inputs, least-privilege containers, and a safe evaluator. The planned jq++ re-implementation in Go is designed not only for performance but also to provide a safe evaluation environment, avoiding shell-based injection risks and ensuring robust execution of declarative expressions.

6.3 Security Considerations

Because jq-front currently relies on shell-based evaluation, incorporating untrusted expressions or includes can pose an injection risk.

We designed the tool and the notation to define a DSL for system configurations or test definitions, where every file is under control and there will be relatively limited security risk. Still, in large corporations, such files are authored by many divisions and we need to consider user provided files untrusted. A strategy we recommend today is to simply prohibit evaluation mechanism for user provided files before rendering the entire effective JSON file. That is, when you are building a DSL by JSON+, you fully leverage features such as inheritances and evaluations; then, package them as "built-in" files which provides the DSL’s structure. They are trusted files. On the other hand, for user provided files, there are already reusable builtin files as a part of the DSL. If so, users can simply extend or include those bundled files. By this approach, it will be possible to build a custom DSL pipeline based on JSON++ used by many divisions.

We recognize this is the area where the tool needs further enhancements such as an introduction of `-safe` option.

6.4 Summary

In summary, jq-front positions itself not as a more human-friendly JSON, but as a structural extension of JSON designed to be composable, minimal, and faithful to UNIX pipelines. Its contribution lies in introducing JSON++ as a lightweight mechanism for inheritance and evaluation while keeping full compatibility with the JSON ecosystem. Performance improvements and tooling support are left for the forthcoming jq++ re-implementation, while current limitations remain secondary to the broader design philosophy.

7 Related Work

Several configuration and data serialization notations have been proposed and widely adopted in practice, each with different emphases on human-readability, structural expressiveness, and compatibility with JSON. YAML, TOML, and HCL extend JSON by introducing syntactic conveniences that aim to make configuration files easier for humans to read and write. While this has improved usability in many domains, these approaches tend to couple readability with structural definition, sometimes at the expense of predictability or machine processing simplicity.

HOCON (Human-Optimized Config Object Notation) represents a more direct attempt to address redundancy and expressiveness limitations in JSON. It supports object merging, includes, substitutions, and other mechanisms that ease the burden of writing large configuration files. jq-front is conceptually related in this regard, since it also addresses the problem of repetitive definitions in structured data. However, jq-front’s approach differs fundamentally in that it separates two concerns: (1) structural expressiveness (e.g., object extension, inheritance-like composition) and (2) human readability. jq-front prioritizes the first and deliberately leaves the second to downstream tools, aligning more closely with the UNIX philosophy of “do one thing well.”[17]

An important consequence is that by defining JSON++, we automatically obtain YAML++, TOML++, and HCL++ as well. Because these formats are either natively compatible with JSON or easily converted to it, they can all benefit from the same abstraction mechanisms without new parsers or syntactic extensions. This contrasts with HOCON, which introduces its own ecosystem-specific notation and parsing model. JSON++ therefore acts as a unifying abstraction layer for the entire JSON family, delivering expressive power without fragmenting the tool landscape.

Other configuration management ecosystems also provide features similar to object extension and value interpolation. For instance, Helm charts in Kubernetes allow templating and parameter substitution, while Ansible and Terraform rely on YAML or HCL enriched with their own interpolation semantics. These systems, however, are tightly coupled to their respective platforms. jq-front, by contrast, remains format-agnostic and lightweight, enabling its integration into diverse pipelines wherever JSON is the interchange format.

Compared to these alternatives, jq-front emphasizes predictable semantics over convenience-driven syntactic sugar. Performance, at present, is constrained by its implementation language, but the planned re-implementation in a high-performance language such as Java or Go is expected to broaden its applicability.

In summary, jq-front can be situated in the landscape of configuration notations as a tool that complements rather than competes with formats like YAML, TOML, HOCON, or HCL. By offering structural expressiveness grounded in JSON compatibility, jq-front provides a bridge between the simplicity of JSON and the redundancy-reducing mechanisms often embedded in higher-level configuration systems.

8 Conclusion and Demo Plan

In this work, we introduced jq-front, a lightweight command-line tool that extends JSON with declarative inheritance and expression evaluation. By separating human readability from structural expressiveness, jq-front offers a pragmatic solution for reducing redundancy and improving maintainability of JSON-family based configurations. This design follows the UNIX philosophy of “do one thing well,” enabling composition of small, expressive operations within existing pipelines rather than introducing a new monolithic syntax.

Beyond the tool itself, this work contributes JSON++ as a lightweight extension of JSON. JSON++ preserves full JSON validity while adding two minimal constructs: inheritance (`$extends`, `$includes`) and expression evaluation (for both values and keys). This combination allows developers to express complex structures concisely without breaking compatibility with existing JSON tooling. jq-front serves as the current reference implementation of JSON++, demonstrating its practicality across heterogeneous environments.

The main limitation of jq-front today is performance, stemming from its layered use of Bash and jq. Although sufficient for moderate-scale use, larger workloads expose overheads that motivate a reimplement in a high-performance language such as Java or Go. This future version, to be released under the name jq++, will provide improved performance, portability, and robustness while preserving the design principles established by jq-front.

In summary, JSON++ is not merely “YAML++” or “TOML++,” but a unifying abstraction layer that enhances all JSON-compatible formats with concise reuse and dynamic evaluation. By replacing fragmented, ecosystem-specific enhancements with a minimal and uniform mechanism, JSON++ serves as a remedy to the current landscape of *ad hoc* solutions (as discussed in Section 1.1). jq-front, as its reference implementation, demonstrates that such a unification can be realized without inventing new notations or breaking compatibility, thereby offering developers a consistent foundation for maintainable and expressive configurations.

We plan to demonstrate jq-front in real-world scenarios where JSON-family notations play a central role in configuration management. The demo will highlight the two key features of JSON++ and jq-front:

- Inheritance and composition of JSON files for reducing duplication across configurations.
- Dynamic evaluation of node values and keys for parameterization and context-sensitive structures.
- Integration into a UNIX pipeline, showing interoperability with existing tools and notations such as jq and YAML.
- Handling of edge cases, including missing files or complex JSON path references.

These demonstrations will illustrate how JSON++ extends the expressive power of JSON while preserving its compatibility. Through jq-front today, and jq++ in the near future, we aim to provide developers with a uniform, minimal, and composable approach to structured configuration management.

Acknowledgement

The author acknowledges the assistance of OpenAI’s ChatGPT (GPT-5) for language refinement and structural organization. All conceptual content and final decisions are the author’s own.

Appendix: Practical Example

So far, we have introduced compact and clean examples to clarify the mechanisms of JSON++ (inheritance and evaluation). In practical settings, however, configurations tend to be verbose and repetitive. This appendix illustrates a more realistic case to show how JSON++ and jq-front reduce redundancy at scale.

Baseline (Without JSON++)

The following excerpt shows part of a configuration set for a search service. Each environment (sandbox, staging, production) repeats the same blocks with only minor variations. For readability, the middle sections are omitted.

```

1 {
2   "search-api": {
3     "image": "ghcr.io/example/search-api:1.1.0-rc.2",
4     "resources": {
5       "requests": { "cpu": "200m", "memory": "256Mi" },
6       "limits":    { "cpu": "500m", "memory": "512Mi" }
7     },
8     "env": {
9       "API_URL": "https://api-staging.example.com"
10    }
11  },
12  "storage-api": {
13    "image": "ghcr.io/example/storage-api:1.1.0-rc.2",
14    "resources": {
15      "requests": { "cpu": "200m", "memory": "256Mi" },
16      "limits":    { "cpu": "500m", "memory": "512Mi" }
17    },
18    "env": {
19      "STORAGE_API_TOKEN": "***staging-token***"
20    }
21  },
22  ...
23  // 120 similar lines omitted
24  ...
25 }
```

This file is long and cluttered, with many blocks duplicated across services and environments. In production deployments, such duplication often grows to hundreds of lines.

Refactored (With JSON++)

Using JSON++, the same configuration can be expressed much more concisely:

```

1 {
2   "$local": {
3     "baseResources": {
4       "resources": {
5         "requests": { "cpu": "200m", "memory": "256Mi" },
6         "limits":    { "cpu": "500m", "memory": "512Mi" }
7       }
8     }
9   },
10  "search-api": {
11    "$extends": ["baseResources"],
12    "image": "ghcr.io/example/search-api:1.1.0-rc.2",
13    "env": { "API_URL": "https://api-staging.example.com" }
14  },
15  "storage-api": {
```

```

16     "$extends": ["baseResources"],
17     "image": "ghcr.io/example/storage-api:1.1.0-rc.2",
18     "env": { "$includes": ["staging/storage-api.credentials.json"] }
19   }
20 }

```

, where a file `staging/storage-api.credentials.json` has the following content.

```

1 { "STORAGE_API_TOKEN": "***staging-token***" }

```

Here, the duplicated `resources` block is factored out once under `$local`. Both services simply extend from it, reducing duplication and improving maintainability.

Quantitative Comparison

	Baseline JSON	JSON++ with jq-front
Lines of configuration	140	49
Repeated resource blocks	6	0

Discussion

This example demonstrates the scalability of JSON++:

- Repetition across services and environments collapses into compact definitions.
- Changes to shared parameters (e.g., CPU/memory requests) are made in one place.
- The output remains plain JSON, preserving compatibility with existing pipelines.

In summary, while small toy examples show the mechanics of JSON++, large-scale cases like this appendix reveal its true benefit: maintaining concise, consistent, and less error-prone configurations across environments.

References

- [1] Douglas Crockford. *The application/json Media Type for JavaScript Object Notation (JSON)*. IETF RFC 4627. 2006. URL: <https://www.rfc-editor.org/rfc/rfc4627>.
- [2] T. Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. IETF RFC 8259. 2017. URL: <https://www.rfc-editor.org/rfc/rfc8259>.
- [3] Oren Ben-Kiki, Clark Evans, and Ingy döt Net. *YAML Ain't Markup Language (YAML) Version 1.2*. Available at <https://yaml.org/spec/1.2/spec.html>. 2009.
- [4] Tom Preston-Werner. *TOML: Tom's Obvious, Minimal Language*. Available at <https://toml.io/en/>. 2013.
- [5] HashiCorp. *HashiCorp Configuration Language (HCL)*. Available at <https://github.com/hashicorp/hcl>. 2014.
- [6] HashiCorp. *Terraform by HashiCorp*. <https://www.terraform.io/>. Accessed: 2025-09-06. 2014.
- [7] The Consulting CTO. "Working with Terraform: 10 Months In". In: *The Consulting CTO blog* (Oct. 2017). Describes how writing over 40,000 lines of HCL exposed issues of code duplication, low composability, and the difficulty of maintaining Terraform configurations. URL: <https://theconsultingcto.com/posts/working-with-terraform-10-months-in/>.
- [8] RedHat. *Ansible by RedHat*. <https://docs.ansible.com/>. Accessed: 2025-09-06. 2012.
- [9] Armin Ronacher. *Jinja2: A Modern and Designer-Friendly Templating Language for Python*. <https://palletsprojects.com/p/jinja/>. Accessed: 2025-09-06. 2008.
- [10] Stack Overflow user larsks (with SNR follow-up). *How to reuse repetitive "get version" tasks across multiple Ansible roles?* <https://stackoverflow.com/questions/68165832/ansible-how-to-use-tasks-multiple-times-in-different-roles>. Demonstrates how practitioners cope with repetition via ad-hoc reuse patterns (include_role, import_role, dependencies). 2021.
- [11] Lightbend. *HOCON: Human-Optimized Config Object Notation*. Available at <https://github.com/lightbend/config/blob/main/HOCON.md>. 2012.

- [12] Brian W. Kernighan and Rob Pike. *The Unix Programming Environment*. Englewood Cliffs, NJ: Prentice Hall, 1984. ISBN: 0-13-937681-X.
- [13] The Helm Authors. *Helm: The Kubernetes Package Manager*. <https://helm.sh/>. Accessed: 2025-09-10. 2015.
- [14] Hiroshi Ukai. *jq-front: A lightweight JSON preprocessor with inheritance and evaluation*. <https://github.com/dakusui/jq-front>. Accessed: 2025-09-07. 2019.
- [15] Jsonnet Authors. *Jsonnet Specification*. <https://jsonnet.org/ref/spec.html>. Accessed 2025-09-11.
- [16] The CUE Authors. *The CUE Language Specification*. <https://cuelang.org/docs/reference/spec/>. Accessed 2025-09-11. 2025.
- [17] Eric S. Raymond. *The Art of UNIX Programming*. Addison-Wesley, 2003. ISBN: 978-0131429017.